Transform Documentation into Insights with AI

Scenario

Security teams deal with a steady flow of documentation: advisories, best-practice guides, internal policies, architecture notes, and long technical write-ups. These materials often contain essential details, but they're time-consuming to read closely. AI can help reduce that burden by condensing information, highlighting important points, and making large documents faster to work through.

This lab looks at how language models handle security-focused material and what influences the quality of their summaries. You'll explore how clarity, structure, and context shape the output, and how different approaches to summarization reveal different aspects of a document.

Your Mission

- Learn how AI interprets security documentation and why structured prompts matter.
- Build a summary workflow that turns long text into clear, usable insights.
- Turn that workflow into a reusable custom model anyone can use.
- Use an automated pipeline to enrich and transform security content at scale.

5.2.1 Basic Summarization

Summarization is one of the tasks language models handle most reliably, and it's something people turn to AI for every day. Security work involves a steady stream of written material, so being able to quickly condense a document into its essential points is genuinely useful. In this first exercise, you'll work with a short plaintext file to see how the model approaches summarization in a simple setting. This gives you a baseline for understanding how the model interprets content and what influences the clarity and accuracy of its summaries.

Step 1: Download the files in Labfiles5.2 from the GitHub repository. Move it to **/home/kali/Labfiles5.2**.

Step 2: Open **https-encryption.md** inside Labfiles5.2 and review the file.

> **Insights:** The document gives a brief overview of why HTTPS matters for OpenWebUI, noting its role in protecting data and enabling features like Voice Calls. It also outlines common options for enabling HTTPS, such as load balancers, reverse proxies, Cloudflare, or ngrok.

Step 3: From the desktop, open **Firefox**. Open and sign into **Open WebUI**.

Step 4: From Open WebUI, start a new chat with **GPT OSS 20B**.

Step 5: Beneath the chat box, select **+ More** and then select **Upload Files**.

Step 6: Select **https-encryption.md** from **/home/kali/Labfiles5.2**, then select **Open**.

Step 7: In the chat, enter the following prompt and note the response:

```
Prompt 1:
Summarize this file.
```

> **Insights:** A prompt like "summarize this file" gives the model almost no direction, so the result is usually brief and shallow. It works for a quick glance, but it often misses the important details or structure of the document.

Step 8: In the same chat, enter the following prompt and note the response:

```
Prompt 2:
Summarize the document with the following sections:
Purpose of HTTPS
Why HTTPS is required for OpenWebUI
Environments where HTTPS is especially important
Deployment options (brief)
Key takeaways

Only use information from the document.
```

Insights: Using a more detailed prompt produces a noticeably stronger summary. By asking for specific sections, the model knows what to focus on and returns information that's clearer, more complete, and aligned with what you actually care about. This difference becomes even more significant with longer or more complex documents, where unstructured prompts tend to miss important details or collapse everything into a vague overview.

Step 9: In the same chat, enter the following prompt and note the response:

```
Prompt 3:
I'm trying to justify implementing HTTPS to management. Please give me a
detailed summary I can use for this.
```

Insights: A management-focused summary emphasizes the higher-level points that support a decision, rather than covering each part of the document in detail. It's useful for presenting impact and justification, but it isn't as thorough as a structured, section-based prompt and may leave out smaller points that were captured earlier.

Step 10: Select **Workspace** from the left menu, then select the **Knowledge** tab.

Step 11: Select the **Data** collection, then select **+** to the right of the page and choose **Upload files**.

Step 12: Select **Exchange_Server_Best_Practices.pdf** from **/home/kali/Labfiles5.2** and select **Open**.

Step 13: In a new chat with **GPT OSS 20B**, enter # to bring up the knowledge store. Select **Exchange_Server_Best_Practices.pdf** from the list.

Insights: This file is much larger than the previous one and requires chunking/retrieval through the model's knowledge base. It outlines key hardening measures for on-premises Microsoft Exchange Server environments. It emphasizes timely patching, minimizing attack surfaces, strong authentication, and encryption to reduce exposure to targeted threats.

Step 14: In the chat, enter the following prompt and note the response:

```
Prompt 4:
Summarize the document's recommendations for maintaining a prevention
posture.
```

Step 15: In the same chat, enter the following prompt and note the response:

```
Prompt 5:
Summarize what this document says about authentication and encryption
hardening for Exchange Server.
```

Step 16: In the same chat, enter the following prompt and note the response:

```
Prompt 6:
Summarize the document's advice for handling end-of-life Exchange Server.
```

Insights: Because this PDF lives in the knowledge base, the model can pull just the portions that relate to each question instead of trying to process the entire document at once. This makes the responses more focused and grounded, even when the source is long and detailed. Retrieval gives the model the structure it needs to handle larger security documents effectively.

5.2.2 Building a Summary Chain

In the previous exercise, you explored different ways to summarize security documentation — from simple one-line prompts to section-based requests and targeted extractions through the knowledge base. Now you'll take the next step: turning that ad-hoc prompting into a repeatable, structured workflow.

Security teams often need consistent outputs from long or complex documents, whether they're reviewing vendor hardening guides, policy updates, or internal configuration standards. Instead of reinventing your prompt every time, you can build a summary chain: a sequence of short prompts that progressively refine raw text into something actionable.

In this exercise, you'll walk through a multi-step summary chain, observe how each stage transforms the document, and then combine the chain into a single system prompt. Finally, you'll package it as a reusable custom model in Open WebUI — giving you a ready-to-use tool for future summaries and analysis.

Step 1: Select **Workspace** from the left menu, then select the **Knowledge** tab.

Step 2: Select the **Data** collection, then select **+** to the right of the page and choose **Upload files**.

Step 3: Select **Exchange_Server_Best_Practices.pdf** from **/home/kali/Labfiles5.2** and select **Open**. If already uploaded from the previous section, skip this step.

Step 4: In a new chat with **GPT OSS 20B**, enter # to bring up the knowledge store. Select **Exchange_Server_Best_Practices.pdf** from the list.

Step 5: In the chat, enter the following prompt and note the response:

```
Prompt 1:
Identify the major security categories emphasized in this document. List
only the category names.
```

Insights: Starting with categories gives structure to your entire chain. It helps you avoid summaries that feel scattered or uneven, especially when the source material is long.

Step 6: In the same chat, enter the following prompt and note the response:

```
Prompt 2:
For each category you listed, provide a 2-3 sentence summary capturing the
document's key guidance.
```

Insights: This turns the category outline into a useful middle layer — long enough to be meaningful, but still concise enough to guide the next steps.

Step 7: In the same chat, enter the following prompt and note the response:

```
Prompt 3:
Convert the category summaries into an actionable hardening checklist.
Each item must be specific, testable, and grouped under the appropriate
category.
```

Insights: Security teams need checklists that can be assigned, tracked, and validated. This step forces the model to translate guidance into tasks you could actually implement.

Step 8: In the same chat, enter the following prompt and note the response:

```
Prompt 4:
From the full checklist above, identify the top 5 highest-impact actions a
security engineer should take first. Include a one-sentence justification
for each.
```

Insights: Prioritization is where summarization becomes decision support. By adding justification, the model exposes why each action matters.

Step 9: In the same chat, enter the following prompt and note the response:

```
Prompt 5:
Rewrite the prioritized actions as a management briefing focused on
business risk, operational impact, and justification for investment.
```

Insights: Different roles need different framing. Management summaries emphasize risk and outcomes rather than configuration-level detail.

Step 10: From Open WebUI, select **Workspace** from the left menu, then select the **Models** tab.

Step 11: From the Models tab, select **+** to the right of the page to create a new model.

Step 12: On the model creation page, set the following settings:

Setting	Value
Model Name	Security Summarizer
Base Model (From)	GPT OSS 20B
Description	Provides structured security document summaries
Visibility	Public

Step 13: For the **System Prompt**, enter the following:

```
You are a Security Analysis Summarization Assistant.
When given any security document (plaintext, PDF, or KB-sourced), follow
this workflow:

1. Identify major security categories in the document.
2. Summarize each category in 2-3 sentences.
3. Convert the summaries into a specific, testable hardening checklist
grouped by category.
4. Prioritize the top 5 highest-impact tasks, with brief justification.
5. Rewrite the prioritized list as a management-facing summary focused on
risk and operational impact.

Return results with clear section headers and no extra commentary.
```

Step 14: Under the **Knowledge** section, select **Select Knowledge** and select the **Data** collection.

Step 15: Ensure the model settings are correct, then select **Save & Create** at the bottom.

> **Insights:** Creating a custom model turns your summary chain into a reusable tool. Instead of relying on someone knowing the right prompts or understanding prompt engineering, the model itself enforces the workflow. Anyone — regardless of experience — can give it a simple request and still get the same structured, reliable analysis you built during this exercise.

Step 16: Start a new chat with the **Security Summarizer** model you just created.

Step 17: In the chat, enter # to bring up the knowledge store. Select **Exchange_Server_Best_Practices.pdf** from the list.

Step 18: With the file selected, enter the following prompt and note the response:

```
Prompt 1:
Please provide a security analysis summary of this document.
```

Insights: Because the summary chain is now part of the model's system prompt, even a short request triggers the full structured workflow. You no longer need detailed prompts — the model applies the process automatically, giving you consistent, high-quality summaries from minimal input.

Action: Validate model creation by selecting the verify button in the lab interface.